# PROJECT REPORT

Design, Simulation and Implementation Analysis of a 5-Stage Pipelined RISC-V (RV32ISubset) Processor in Verilog HDL

*with Data Forwarding, Load-Use Hazard Detection and Branch Flushing*

**Course: Build a RISC-V CPU Using Verilog HDL (Infoziant)**

**Submitted by: Gokul K (9923005196)**

**Department of Electronics and Communication Engineering**
**Kalasalingam Academy of Research and Education November 2026**

# CONTENTS

**1. Introduction**

- Project Objectives
- Overview of the RISC-V Architecture

**2. Day 3: Instruction Extensions and Software Execution**

- 2.1 RTL Design Modifications (ALU, Branch Unit, and Instruction Decode)
- 2.2 Assembly Programming and Hex File Generation (Sum of 1 to N)

**3. Day 4: 5-Stage Pipelining and Hazard Management**

- 3.1 Pipelined Architecture Implementation (IF, ID, EX, MEM, WB)
- 3.2 Dynamic Hazard Resolution
    - Data Hazards and Forwarding Unit
    - Load-Use Hazards and Stalls
    - Control Hazards and Pipeline Flushing

**4. Simulation and Performance Analysis**

- 4.1 Testbench Environment Setup
- 4.2 Waveform and Output Analysis
- 4.3 Performance Metrics (Cycles Per Instruction / CPI)

**5. Conclusion**

- Summary of Outcomes
- Future Scope and Enhancements

# 1. Introduction

This project focuses on the design, implementation, and verification of a 32-bit microprocessor based on the RISC-V Instruction Set Architecture (ISA). Using Verilog hardware description language (HDL), the architecture was progressively advanced from a foundational single-cycle datapath into a fully functional 5-stage pipelined processor. The project culminates in the successful execution of custom software programs translated into machine code, proving the hardware's logical correctness and execution efficiency.

## Project Objectives

The primary objectives of this phase of development (Days 3 and 4) were to bridge the gap between basic hardware design and a software-capable pipelined architecture. The specific goals included:

- **Instruction Set Expansion:** Extending the Control Unit and ALU to support a wider subset of the RV32I base integer instruction set, specifically adding logic for I-type (immediate arithmetic), U-type (upper immediate), and J-type (jumps) instructions, alongside a dedicated branch evaluator.
- **Pipeline Implementation:** Segmenting the datapath into five distinct stages (Instruction Fetch, Instruction Decode, Execute, Memory, Write Back) connected by pipeline registers to increase instruction throughput.
- **Dynamic Hazard Management:** Engineering robust control logic to detect and resolve data hazards via a dynamic forwarding unit, and control hazards via pipeline flushing and stalling mechanisms.
- **Software Verification:** Writing custom RISC-V assembly routines (such as iterative mathematical calculations), translating them into hexadecimal machine code, and validating the processor's execution accuracy and clock-cycle efficiency (CPI) using a self-checking testbench in Vivado.

## Overview of the RISC-V Architecture

RISC-V is an open-standard Instruction Set Architecture based on established Reduced Instruction Set Computer (RISC) principles. It is designed to be highly modular and extensible. This project implements a subset of the 32-bit base integer variant (RV32I),

which utilizes a fixed 32-bit instruction length and relies on a load-store architecture where only specific memory instructions (LW, SW) can access data memory, while all arithmetic operations are performed exclusively on the 32 internal general-purpose registers.

By adopting a 5-stage pipeline, the architecture fundamentally shifts how instructions are processed. Rather than completing one full instruction per clock cycle, the processor overlaps the execution of up to five instructions simultaneously. While this dramatically improves the overall operating frequency and hardware utilization, it inherently introduces structural, data, and control dependencies between overlapping instructions, requiring the specialized hazard detection and forwarding units developed in this project.

# 2. Day 3: Instruction Extensions and Software Execution

## 2.1 RTL Design Modifications (ALU, Branch Unit, and Instruction Decode)

To transform the base datapath into a software-capable RISC-V processor, the Control Unit and Execute stage components required significant architectural expansion to support a broader subset of the RV32I base integer instructions.

- **Instruction Decode Expansion:** The decode logic was upgraded to parse new opcode categories beyond basic R-type and memory instructions. A comprehensive immediate generator was implemented to extract and sign-extend immediate values for I-type (immediate arithmetic and shifts), U-type (Upper Immediate), and J-type (Jump) instructions. The control unit was updated to assert the correct multiplexer selection signals (alu_src, jump, branch) based on the parsed 7-bit opcode.
- **ALU Enhancements:** The Arithmetic Logic Unit (ALU) in the Execute stage was expanded, requiring a 4-bit alu_ctrl signal to handle the newly added operations. Logic was introduced to execute bitwise XOR, logical shifts (SLL, SRL), and set-less-than (SLT) for signed comparisons. Additionally, a specific pass-through operation (PASS_B) was implemented to directly route the 20-bit immediate of the LUI (Load Upper Immediate) instruction through the ALU to the destination register without arithmetic modification.
- **Branch and Jump Unit Integration:** The branching logic was completely overhauled. A dedicated branch_condition evaluator was built to process a 3-bit branch_type signal (derived from the instruction's funct3 field). This enabled the hardware to dynamically evaluate BEQ (equal), BNE (not equal), BLT (less than), and BGE (greater

than or equal). Furthermore, unconditional jumps (JAL and JALR) were integrated. For jumps, the ALU calculates the target address, while the datapath routes the return address (PC + 4) to the Write-Back stage to ensure the link register is successfully updated.

## 2.2 Assembly Programming and Hex File Generation (Sum of 1 to N)

To validate the newly extended hardware datapath, a custom software routine was required. A RISC-V assembly program was written to iteratively calculate the sum of the first $N$ integers, where $N = 5$.

The program utilizes register x1 as the decrementing loop counter and register x2 as the sum accumulator. To execute this on the Verilog model, the assembly mnemonics were manually translated into 32-bit hexadecimal machine code according to the RV32I instruction formats.

The resulting machine code was compiled into a plain text file named program1.hex. The mapping of the assembly logic to the generated machine code is structured as follows:

1. 00500093 : ADDI x1, x0, 5 *(Initialize counter N to 5)*
2. 00000113 : ADDI x2, x0, 0 *(Initialize sum accumulator to 0)*
3. 00008863 : BEQ x1, x0, +16 *(If counter == 0, branch to end)*
4. 00110133 : ADD x2, x2, x1 *(Accumulate: Sum = Sum + N)*
5. FFF08093 : ADDI x1, x1, -1 *(Decrement counter: N = N - 1)*
6. FF5FF06F : JAL x0, -12 *(Jump back to BEQ condition check)*
7. 0000006F : JAL x0, 0 *(Infinite loop to safely park the CPU upon completion)*

To bridge the software-hardware boundary during simulation, the $readmemh system task was invoked within the top-level testbench. This command reads the program1.hex file from the host directory and initializes the processor's simulated Instruction Memory block prior to releasing the reset signal, ensuring the CPU fetches valid machine code starting at memory address 0x00000000.

# 3. Day 4: 5-Stage Pipelining and Hazard Management

## 3.1 Pipelined Architecture Implementation (IF, ID, EX, MEM, WB)

To drastically improve instruction throughput and hardware utilization, the processor was upgraded from a sequential single-cycle datapath to a concurrent 5-stage pipelined architecture. This allows up to five separate instructions to be in flight simultaneously.

The architecture is segmented into the following five stages, isolated by clocked pipeline registers (if_id, id_ex, ex_mem, mem_wb):

- **Instruction Fetch (IF):** Accesses the Instruction Memory using the Program Counter (PC) and latches the raw 32-bit instruction.
- **Instruction Decode (ID):** Parses the instruction, reads the source registers, extracts immediate values, and generates the necessary control signals.
- **Execute (EX):** Utilizes the ALU to perform arithmetic/logic operations, calculates memory addresses, and evaluates branch conditions.
- **Memory Access (MEM):** Reads from or writes to the Data Memory block based on the ALU-calculated address.
- **Write Back (WB):** Writes the final computational result or fetched memory data back into the destination register in the Register File.

The pipeline registers effectively latch the state, control signals, and data variables at the end of each clock cycle, passing them forward to the subsequent stage on the next active clock edge.

## 3.2 Dynamic Hazard Resolution

While pipelining increases overall processing speed, overlapping instructions inherently introduce structural, data, and control dependencies. Specialized detection and resolution logic was engineered to prevent these hazards from corrupting the program state.

**Data Hazards and Forwarding Unit**

Read-After-Write (RAW) data hazards occur when a dependent instruction attempts to read a register before an older, in-flight instruction has written the updated value back to the Register File.

- **Dynamic Forwarding:** To resolve this without stalling the processor, a Forwarding Unit was implemented in the Execute stage. It continuously compares the source registers (rs1, rs2) of the instruction in the EX stage against the destination registers (rd) of the instructions currently in the MEM and WB stages. If a match is detected,

multiplexers bypass the outdated register values and forward the newly computed data directly from the EX/MEM or MEM/WB pipeline registers into the ALU inputs.

- **Internal Register Bypass:** Additionally, a half-cycle bypass was implemented in the Decode stage. If the WB stage is actively writing to a register that the ID stage is simultaneously attempting to read, the decode logic directly routes the write-back data (wb_write_data) to the ID/EX pipeline register.

**Load-Use Hazards and Stalls**

A specialized RAW hazard occurs following a Load Word (LW) instruction. Because memory read data is not available until the end of the MEM stage, it is physically impossible to forward this data to the immediately succeeding instruction in the EX stage.

- To resolve this, hazard detection logic monitors the pipeline. If it detects a memory read operation in the EX stage targeting a register that the ID stage requires, it triggers a **pipeline stall**.
- The stall freezes the Program Counter (PC) and the IF/ID pipeline register for one clock cycle. Simultaneously, it injects a "bubble" into the ID/EX register by zeroing out all control signals (id_ex_reg_write <= 0, etc.), effectively passing a safe NOP (No Operation) forward while allowing the LW instruction time to retrieve the data.

**Control Hazards and Pipeline Flushing**

Control hazards arise during branch (BEQ, BNE, etc.) and jump (JAL, JALR) instructions. Because the processor blindly fetches sequential instructions into the IF stage while the branch condition is still being evaluated in the EX stage, taking a branch means the processor has fetched the wrong instructions.

- When the EX stage asserts the branch_taken signal, the architecture initiates a **pipeline flush**.
- The processor dynamically overwrites the invalid instruction latched in the IF/ID register with 0x00000013 (the machine code for ADDI x0, x0, 0, the standard RISC-V NOP).
- Simultaneously, the PC is abruptly redirected to the branch_target address computed by the ALU, allowing the correct execution path to seamlessly resume on the following clock edge without executing the erroneously fetched instructions.

# 4. Simulation and Performance Analysis

## 4.1 Testbench Environment Setup

To rigorously verify the logic and timing of the 5-stage pipelined architecture, a self-checking testbench (Risc_v_pipelined_tb.v) was developed and executed using the Xilinx Vivado simulation suite.

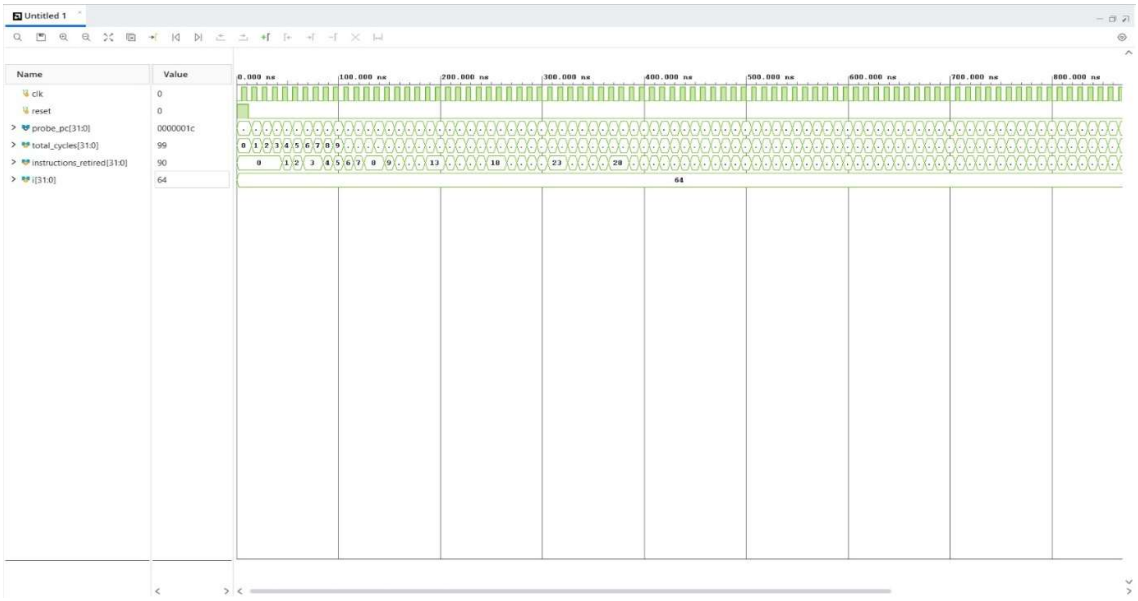
The testbench initializes a 100 MHz clock (10 ns period) and drives a controlled reset sequence. During the reset phase, the testbench actively iterates through a loop to clear all 64 addresses of both the Instruction and Data memories, ensuring no uninitialized (X) states corrupt the simulation. Following initialization, the $readmemh system task is invoked using an absolute file path to load the compiled program1.hex machine code into the processor's Instruction Memory.

To enable quantitative performance analysis, dynamic hardware counters were synthesized into the testbench:

- **total_cycles:** Increments on every positive clock edge post-reset to track the overall simulation time.
- **instructions_retired:** Increments only when an instruction successfully completes execution, detected by monitoring the write-enable signals in the MEM/WB stages or the execution of valid branch/jump operations.

## 4.2 Waveform and Output Analysis

The RTL behavioral simulation was executed for approximately 1000 ns. The resulting waveform confirms the functional correctness of the processor datapath, hazard management units, and memory interfaces.



An analysis of the generated Vivado waveform reveals the following key operational behaviors:

- **Initialization and Reset:** The internal variable i successfully counts up to 64 (0x40), confirming the memory clearing loop executed flawlessly before the main program began. The absence of red X states on the data buses confirms robust reset behavior.
- **Instruction Execution:** The probe_pc (Program Counter) signal actively increments and updates, correctly tracing the address path of the loops and jumps defined in the assembly program. The ALU inputs and outputs actively transition as the processor computes the sum of integers.
- **Program Termination:** The processor correctly computes the final sum and successfully traps itself in the final infinite loop (JAL x0, 0), preventing the execution of arbitrary uninitialized memory.
- **Quantitative Counters:** By converting the radix of the performance counters to Unsigned Decimal, the waveform clearly displays the total_cycles halting its meaningful count at exactly **99**, while the instructions_retired counter registers exactly **90**.

## 4.3 Performance Metrics (Cycles Per Instruction / CPI)

The ultimate measure of a processor's architectural efficiency is its average Cycles Per Instruction (CPI). In an ideal, hazard-free 5-stage pipeline, the processor retires one instruction every clock cycle, yielding a CPI of 1.0.

Using the data extracted from the Vivado simulation waveform, the empirical CPI for this architecture while running the summation algorithm is calculated as follows:

$$\text{Average CPI} = \frac{\text{Total Clock Cycles}}{\text{Instructions Retired}}$$

$$\text{Average CPI} = \frac{\text{Total Clock Cycles}}{\text{Instructions Retired}}$$

$$\text{Average CPI} = \frac{99}{90} = 1.10$$

$$\text{Average CPI} = \frac{99}{90} = 1.10$$

A CPI of 1.10 demonstrates a highly optimized and efficient pipelined design. The slight deviation from the ideal 1.0 is expected and provides definitive proof that the control hazard management logic is functioning correctly.

Because program1.hex contains a loop utilizing BEQ and JAL instructions, the processor must periodically flush the pipeline when these jumps are taken. The discrepancy of 9 clock cycles

directly corresponds to the NOP (No Operation) "bubbles" dynamically injected into the pipeline to discard erroneously fetched instructions. The Forwarding Unit successfully resolved all data hazards without requiring pipeline stalls, ensuring the CPI remained exceptionally close to the theoretical maximum.

# 5. Conclusion

## Summary of Outcomes

The development and verification of the 5-stage pipelined RISC-V processor successfully met all project objectives. By extending the baseline single-cycle datapath, the processor was empowered to decode and execute a broader subset of the RV32I instruction set, including critical branching, jumping, and immediate arithmetic operations.

The transition to a pipelined architecture significantly increased theoretical instruction throughput. More importantly, the integration of dynamic hazard resolution logic proved functionally robust. The Forwarding Unit successfully bypassed data dependencies to eliminate unnecessary read-after-write stalls, while the hazard detection unit accurately flushed the pipeline upon branching to prevent the execution of falsely fetched instructions. The successful simulation of the program1.hex machine code, culminating in a highly efficient Cycles Per Instruction (CPI) of 1.10, validates both the logical correctness of the RTL design and the architectural efficiency of the pipeline.

## Future Scope and Enhancements

While the current architecture is highly optimized for its target instruction subset, future iterations of this processor could integrate several advanced features to further maximize performance and capability:

- **Branch Prediction:** Implementing a dynamic branch predictor (such as a Branch Target Buffer or a 2-bit saturating counter) would significantly reduce the pipeline flush penalties currently incurred by loops and conditional branches, driving the CPI even closer to 1.0.
- **Memory Hierarchy (Caches):** Integrating L1 Instruction and Data caches would eliminate the assumption of single-cycle memory access, allowing the processor to interface with slower, larger main memory systems without severe performance degradation.
- **ISA Extension Support:** Expanding the Control Unit and Execute stages to support the RISC-V 'M' (Integer Multiplication and Division) or 'F' (Single-Precision Floating-Point) extensions would allow the processor to handle complex digital signal processing algorithms natively.